`BackgroundService`, which implements `IHostedService` and is designed to be used for long-running tasks. To create a background task, you must override a single method of this class, `ExecuteAsync()`. You're free to use `async-await` inside this method, and you can keep running the method for the lifetime of your app.

The following listing shows a background service that fetches the latest interest rates using a typed client and saves them in a cache, as you saw in figure 34.1. The `ExecuteAsync()` method keeps looping and updating the cache until the `Cancellation-Token` passed as an argument indicates that the application is shutting down.

Listing 34.2 Implementing a `BackgroundService` that calls a remote HTTP API

```
public class ExchangeRatesHostedService : BackgroundService            ❶
{
    private readonly IServiceProvider _provider;                        ❷
    private readonly ExchangeRatesCache _cache;                         ❸
    public ExchangeRatesHostedService(
        IServiceProvider provider, ExchangeRatesCache cache)
    {
        _provider = provider;
        _cache = cache;
    }

    protected override async Task ExecuteAsync(                         ❹
        CancellationToken stoppingToken)                                ❺
    {
        while (!stoppingToken.IsCancellationRequested)                  ❻
        {
            var client = _provider                                      ❼
                .GetRequiredService<ExchangeRatesClient>();             ❼

            string rates = await client.GetLatestRatesAsync();          ❽
            _cache.SetRates(rates);                                     ❾

            await Task.Delay(TimeSpan.FromMinutes(5), stoppingToken);   ❿
        }
```